

AUTOPROMPT: Eliciting Knowledge from Language Models with Automatically Generated Prompts

Taylor Shin^{*◇} Yasaman Razeghi^{*◇} Robert L. Logan IV^{*◇}

Eric Wallace[♠] Sameer Singh[◇]

[◇]University of California, Irvine [♠]University of California, Berkeley

{tshin1, yrazeghi, rlogan, sameer}@uci.edu
ericwallace@berkeley.edu

Abstract

The remarkable success of pretrained language models has motivated the study of what kinds of knowledge these models learn during pretraining. Reformulating tasks as fill-in-the-blanks problems (e.g., cloze tests) is a natural approach for gauging such knowledge, however, its usage is limited by the manual effort and guesswork required to write suitable prompts. To address this, we develop AUTOPROMPT, an *automated* method to create prompts for a diverse set of tasks, based on a gradient-guided search. Using AUTOPROMPT, we show that masked language models (MLMs) have an inherent capability to perform sentiment analysis and natural language inference without additional parameters or finetuning, sometimes achieving performance on par with recent state-of-the-art supervised models. We also show that our prompts elicit more accurate factual knowledge from MLMs than the manually created prompts on the LAMA benchmark, and that MLMs can be used as relation extractors more effectively than supervised relation extraction models. These results demonstrate that automatically generated prompts are a viable parameter-free alternative to existing probing methods, and as pretrained LMs become more sophisticated and capable, potentially a replacement for finetuning.

1 Introduction

Pretrained language models (LMs) have had exceptional success when adapted to downstream tasks via *finetuning* (Peters et al., 2018; Devlin et al., 2019). Although it is clear that pretraining improves accuracy, it is difficult to determine whether the knowledge that finetuned LMs contain is learned during the *pretraining* or the *finetuning*

process. How can we directly evaluate the knowledge present in pretrained LMs, be it linguistic, factual, commonsense, or task-specific?

Numerous techniques have been proposed to elicit such knowledge by analyzing pretrained LMs’ internal representations. A common strategy is to use probing classifiers—shallow classifiers that predict certain attributes using an LMs’ representations as features (Conneau et al., 2018; Liu et al., 2019). However, probing classifiers require additional learned parameters and are thus susceptible to false positives; high probing accuracy is *not* a sufficient condition to conclude that an LM contains a certain piece of knowledge (Hewitt and Liang, 2019; Voita and Titov, 2020). Attention visualization, another common technique, has a similar failure mode: attention scores may be correlated with, but not caused by the underlying target knowledge, leading to criticism against their use as explanations (Jain and Wallace, 2019; Wiegrefe and Pinter, 2019). Both probing and attention visualizations also struggle to evaluate knowledge that cannot be represented as simple token- or sequence-level classification tasks.

A more direct approach for eliciting knowledge from these models, since they are language models after all, is *prompting*, i.e. converting tasks into a language model format. For example, Radford et al. (2019) frame summarization as a language modeling task by appending “TL;DR:” to the end of an article and then generating from an LM. Similarly, Petroni et al. (2019) manually reformulate a knowledge base completion task as a cloze test (i.e., a fill-in-the-blank problem). Compared to existing model analysis methods, prompting is non-invasive: it does not introduce large amounts of additional parameters or require direct inspection of a model’s representations. Thus prompting pro-

* First three authors contributed equally.

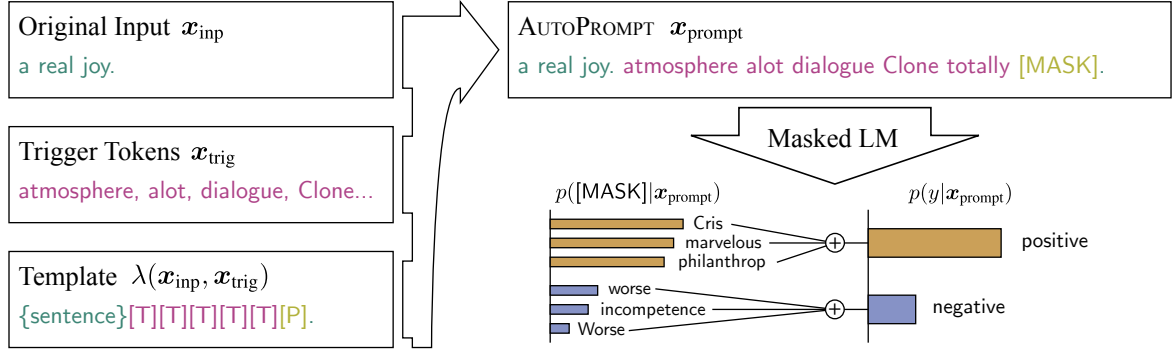


Figure 1: **Illustration of AUTOPROMPT** applied to probe a masked language model’s (MLM’s) ability to perform sentiment analysis. Each input, x_{inp} , is placed into a natural language prompt, x_{prompt} , which contains a single [MASK] token. The prompt is created using a template, λ , which combines the original input with a set of trigger tokens, x_{trig} . The trigger tokens are shared across all inputs and determined using a gradient-based search (Section 2.2). Probabilities for each class label, y , are then obtained by marginalizing the MLM predictions, $p([\text{MASK}] | x_{\text{prompt}})$, over sets of automatically detected label tokens (Section 2.3).

vides a lower bound on what the model “knows”, and is therefore a more useful analysis tool. However, prompting unfortunately requires manually crafting the context to feed into the model. Not only is this time consuming and non-intuitive for many tasks (e.g., textual entailment), more importantly, models are highly sensitive to this context: improperly-constructed contexts cause artificially low performance (Jiang et al., 2020). Overcoming the need to manually specify prompts would make prompting a more widely useful analysis tool.

In this paper, we introduce AUTOPROMPT—an *automated* method for generating prompts for any task, illustrated in Figure 1. Given a task, e.g., sentiment analysis, AUTOPROMPT creates a prompt by combining the original task inputs (e.g. reviews) with a collection of *trigger tokens* according to a template. The same set of trigger tokens is used for all inputs, and is learned using a variant of the gradient-based search strategy proposed in Wallace et al. (2019). The LM predictions for the prompt are converted to class probabilities by marginalizing over a set of associated label tokens, which can either be learned or specified ahead of time, enabling the LM to be evaluated the same as one would any other classifier.

We validate the effectiveness of AUTOPROMPT in numerous experiments. First, we use AUTOPROMPT to construct prompts that test pretrained masked language models (MLMs) on sentiment analysis and natural language inference (NLI). Our tests reveal that, without any finetuning, MLMs perform well on both of these tasks—a properly-

prompted RoBERTa achieves 91% accuracy on SST-2 (better than a finetuned ELMo model (Peters et al., 2018)), and 69% accuracy on a balanced variant of the SICK-E dataset (Marelli et al., 2014). Next, we apply AUTOPROMPT to the fact retrieval tasks of LAMA (Petroni et al., 2019), where we are able to construct prompts that more effectively elicit MLM’s factual knowledge than existing prompts generated using manual and corpus-mining methods. Concretely, we achieve 43.3% precision-at-1, compared to the current best single-prompt result of 34.1% (Jiang et al., 2020). We also introduce a variant of this task, similar to relation extraction (RE), that tests whether MLMs can extract knowledge from a given piece of text. We show that MLMs can actually *outperform* existing RE models when context sentences with real facts are provided, however, they struggle when context sentences are artificially falsified.

Finally, although the goal of AUTOPROMPT is to analyze models, we find that it provides certain practical advantages over finetuning. First, AUTOPROMPT achieves higher average- and worst-case accuracy than finetuning in low-data regimes. Moreover, unlike finetuning, prompting LMs does not require large amounts of disk space to store model checkpoints; once a prompt is found, it can be used on off-the-shelf pretrained LMs. This is beneficial when serving models for multiple tasks.

2 Overview of AUTOPROMPT

A natural way to elicit knowledge from pretrained LMs is to pose tasks as fill-in-the-blank problems.

However, writing prompts is not only time consuming, but it is not clear that the same phrasing will be effective for every model, nor is it clear what criteria determine whether a particular phrasing is the *best* to elicit the desired information. In light of this, we introduce AUTOPROMPT, a method that constructs customized prompts for a specific task and MLM of interest, to cause the MLMs to produce the desired knowledge.¹ An illustration of AUTOPROMPT is provided in Figure 1. The prompt is constructed by taking the original task inputs—a collection of one or more sequences of tokens (e.g., the review in Figure 1)—and mapping them to a sequence of tokens using a template. In the following sections, we describe how AUTOPROMPT uses labeled training data to construct prompts, and how it uses the output of the MLM as a prediction for the task.

2.1 Background and Notation

For the purpose of prompt construction, we distinguish the original task inputs x_{inp} (e.g., the review in Figure 1, “a real joy.”) from the prompt x_{prompt} (e.g., “a real joy. atmosphere alot dialogue Clone totally [MASK].”) that is fed into the MLM. The mapping from x_{inp} to x_{prompt} is performed using a template, λ . This template defines where each input sequence will be placed in the prompt, as well as the placement of any additional tokens. In particular, it must also define the placement of a special [MASK] token for the MLM to fill in (denoted by [P] in the template to distinguish it from other [MASK] tokens that might appear). Feeding the prompt into the MLM produces a probability distribution $p([\text{MASK}]|x_{\text{prompt}})$ describing which tokens most likely fill in the blank.

If class labels naturally correspond to tokens in the vocabulary (e.g., entity names in knowledge base completion tasks), this distribution may be readily interpreted as a distribution over class labels. However, for tasks such as sentiment analysis, there may be a set of label tokens $\mathcal{V}_y$ that correspond to a particular label y . For example, in Figure 1, “Cris”, “marvelous”, and “philanthrop” all indicate positive sentiment. In this case, the class probability is obtained by marginalizing over the

set of label tokens:

$$p(y|x_{\text{prompt}}) = \sum_{w \in \mathcal{V}_y} p([\text{MASK}] = w|x_{\text{prompt}}) \quad (1)$$

2.2 Gradient-Based Prompt Search

So far, we have shown how to reformulate a classification task as a language modeling task using prompts. Here, we propose a method for *automatic prompt construction* based on Wallace et al. (2019). The idea is to add a number of “trigger” tokens that are shared across all prompts (denoted by [T] in the example template in Figure 1). These tokens are initialized to [MASK] tokens, and then iteratively updated to maximize the label likelihood (Equation (1)) over batches of examples.

Formally, at each step, we compute a first-order approximation of the change in the log-likelihood that would be produced by swapping the j th trigger token $x_{\text{trig}}^{(j)}$ with another token $w \in \mathcal{V}$. Then we identify a candidate set $\mathcal{V}_{\text{cand}}$ of the top- k tokens estimated to cause the greatest increase:

$$\mathcal{V}_{\text{cand}} = \text{top-}k \left[w_{\text{in}}^T \nabla \log p(y|x_{\text{prompt}}) \right]_{w \in \mathcal{V}} \quad (2)$$

where w_{in} is the input embedding of w , and the gradient is taken with respect to the input embedding of $x_{\text{trig}}^{(j)}$. Note that computing this candidate set is roughly as expensive as a single forward pass and backward pass of the model (the dot-products require the same amount of multiplications as computing the LM output projection). For each candidate in this set, we then re-evaluate Equation (1) on the updated prompt, and retain the prompt with the highest probability in the next step—this requires k forward passes of the model. An example prompt produced by this method for the task of sentiment analysis is shown in Figure 1.

2.3 Automating Label Token Selection

While in some settings the choice of label tokens is obvious (e.g., when class labels directly correspond to words in the vocabulary), it is less clear what label tokens are appropriate for problems involving more abstract class labels (e.g., NLI). In this section, we develop a general two-step approach to automate the selection of the sets of label tokens $\mathcal{V}_y$. In the first step, we train a logistic classifier to predict the class label using the contextualized embedding of the [MASK] token as input:

$$h = \text{Transformer}_{\text{enc}}(\tilde{x}) \quad (3)$$

¹Although we focus only on MLMs in this work, our method is trivially extendable to autoregressive LMs. The only adjustment is that the predict token must occur at the end of the prompt.